



PROCESADOR DE PALETAS DE COLOR

Documentación Técnica

Materia: Procesamiento Digital de Imágenes

Instituto: IFTS 18

Docente: Juan Ignacio Bonini

Integrantes:

Castañó Alma

Castelao Bravo María del Mar

Chirino Florencia

Fecha: Junio 2026

1. Descripción del Problema

El procesamiento de imágenes digitales implica, entre otras tareas, la extracción y manipulación del color

como elemento fundamental de la información visual. En el campo del diseño, la publicidad y el análisis

visual, la paleta de colores de una imagen define su identidad cromática: transmite emociones, genera

contraste y determina la armonía estética.

El problema abordado en este proyecto consiste en desarrollar una herramienta capaz de:

1. **Extraer automáticamente** los colores dominantes de cualquier imagen digital.
2. **Generar variaciones cromáticas** basadas en relaciones teóricas del círculo cromático (armonías de color).
3. **Aplicar** la paleta seleccionada sobre la imagen original, produciendo una versión recolorada.
4. **Visualizar y comparar** los resultados de forma clara, tanto en consola como en una interfaz web interactiva.

Este problema es relevante en contextos como la edición fotográfica, la generación de identidades

visuales de marca y el análisis estético automatizado.

2. Enfoque y Diseño

La solución se diseñó como una aplicación modular en Python con arquitectura orientada a objetos, que separa responsabilidades en módulos bien definidos. El proyecto expone **dos interfaces de usuario independientes**: una interfaz de línea de comandos (CLI) y una interfaz web interactiva construida con Streamlit.

2.1 Principio de diseño

Cada módulo encapsula una responsabilidad única:

- **Análisis:** extracción de colores dominantes mediante clustering.
- **Generación:** producción de paletas alternativas usando teoría del color.
- **Edición:** aplicación de nuevos colores sobre la imagen.
- **Utilidades:** conversión y clasificación de valores de color.
- **Orquestación CLI:** coordinación del flujo completo desde la entrada del usuario hasta los archivos de salida.
- **Interfaz web:** presentación interactiva en navegador con controles en tiempo real y descarga de resultados.

2.2 Flujo general



CLI: archivos en carpetas de salida

Web: visualización + botones de descarga

3. Técnicas Utilizadas

3.1 Clustering con K-Means

Para extraer los colores dominantes de una imagen se utiliza el algoritmo **K-Means** de la biblioteca scikit-learn. Cada píxel de la imagen se trata como un punto en un espacio tridimensional (R, G, B), y el algoritmo agrupa los píxeles en k clústeres, cuyos centros representan los colores dominantes.

Parámetros: — `n_clusters`: número de colores a extraer (1 a 8, definido por el usuario). — `random_state=42`: semilla fija para reproducibilidad. — La imagen se redimensiona a **200×200 píxeles** antes del clustering para reducir el costo computacional sin afectar la representatividad de la paleta.

3.2 Fusión de colores similares

Tras el clustering, se aplica un filtro de **distancia euclidiana** para eliminar colores casi idénticos. Si la distancia entre dos colores es menor a una tolerancia de 60 unidades, uno de ellos se descarta. Esto evita paletas redundantes.

```
distancia = np.linalg.norm(np.array(color) - np.array(existente))
if distancia < tolerancia:
    agregar = False
```

3.3 Teoría del color: armonías cromáticas

Las paletas alternativas se generan manipulando el canal de tono (Hue) en el espacio de color **HSV** (Hue-Saturation-Value):

Tipo de paleta: Complementaria **Desplazamiento de tono:** +180°

Tipo de paleta: Análoga **Desplazamiento de tono:** -30° y +30°

Tipo de paleta: Triádica **Desplazamiento de tono:** +120° y +240°

La conversión RGB « HSV se realiza con el módulo estándar colorsys de Python.

3.4 Aplicación de paleta por umbral de proximidad

Para reemplazar los colores en la imagen, se utiliza una máscara numpy que identifica píxeles cuyadistancia al color original es menor a **47 unidades** en cada canal:

```
mask = np.all(np.abs(img_array - original) < 47, axis=-1)
img_array[mask] = nuevo
```

3.5 Clasificación de color

Cada color se clasifica automáticamente según dos métricas:

- **Brillo:** promedio de los canales RGB. Clasificado como Oscuro (< 85), Medio (85–170) o Claro (> 170).
- **Temperatura:** comparación entre canal R y canal B. Si $R > B$ - Cálido; si $B > R$ - Frío; si $R = B$ - Neutro.

3.6 Caché de resultados en la interfaz web

La interfaz Streamlit re-ejecuta el script completo ante cada interacción del usuario. Para evitar re-calcular el clustering en cada cambio de slider, la función de extracción de colores está decorada con `@st.cache_data`. El resultado se invalida únicamente cuando cambia la imagen subida o el número de colores solicitados.

4. Implementación

4.1 Estructura de módulos

Módulo: main.py # Punto de entrada CLI

Módulo: paleta_app.py # Orquestador principal (clase PaletaApp)

Módulo: analizador_paleta.py # Carga, clustering y visualización

Módulo: generador_paleta.py # Generación de paletas alternativas

Módulo: editor_imagen.py # Edición y aplicación de paleta

Módulo: color.py # Utilidades: HEX, brillo, temperatura

Módulo: app_streamlit.py # Interfaz web interactiva (Streamlit) -> NUEVO

Módulo: requirements.txt # Dependencias del proyecto

4.2 Descripción de módulos

color.py — **Clase** Color

Encapsula la representación de un color RGB. Métodos principales:

Método rgb_to_hex() → Convierte RGB a string hexadecimal

Método calcular_brillo() → Promedio de canales R, G, B

Método clasificar_brillo() → Categoría: Oscuro / Medio / Claro

Método clasificar_temperatura() → Categoría: Cálido / Frío / Neutro

analizador_paleta.py — **Clase** AnalizadorPaleta

Responsable de la carga y análisis de la imagen.

Método cargar_imagen() → Abre y convierte la imagen a modo RGB

Método extraer_colores() → Aplica KMeans para obtener colores dominantes

Método fusionar_colores_similares(tolerancia) → Elimina colores redundantes por distancia euclidiana

Método mostrar_resultados() → Imprime en consola HEX, temperatura y brillo de cada color

Método obtener_paleta_visual() → Genera imagen PNG con rectángulos de color y datos

Método obtener_paleta_original() → Devuelve la imagen original con recuadros superpuestos

generador_paleta.py — **Clase** → GeneradorPaletas

Transforma la paleta base en variaciones cromáticas.

Método _cambiar_tono(color, delta) → Desplaza el tono en el espacio HSV

Método generar_complementaria() → Aplica desplazamiento de 180°

Método generar_analogos() → Aplica desplazamientos de $\pm 30^\circ$

Método generar_triadicos() → Aplica desplazamientos de 120° y 240°

Método generar_paletas() → Devuelve diccionario con las tres paletas y sus descripciones

editor_imagen.py — Clase EditorImagen

Operaciones de transformación sobre la imagen.

Método `cambiar_brillo(factor)` → Ajusta el brillo con `ImageEnhance.Brightness` (rango 0.1–3.0)

Método `cambiar_saturacion(factor)` → Ajusta la saturación con `ImageEnhance.Color` (rango 0.0–3.0)

Método `cambiar_tamano(ancho, alto)` → Redimensiona la imagen

Método `aplicar_paleta(originales, nuevos)` → Sustituye colores por proximidad (umbral 47)

paleta_app.py — Clase PaletaApp

Orquesta el flujo completo del modo CLI: entrada del usuario - análisis - generación - selección - edición - guardado.

Método `ejecutar()` → Coordina el pipeline completo

Método `_pedir_datos_usuario()` → Solicita ruta de imagen y número de colores por consola

Método `_analizar_imagen()` → Carga la imagen y extrae colores dominantes

Método `_generar_y_guardar_paletas()` → Genera las tres variantes y las guarda en disco

Método `_aplicar_paleta_seleccionada()` → Muestra menú y aplica la paleta elegida

Método `_crear_imagen_comparativa()` → Genera imagen antes/después lado a lado

app_streamlit.py — Interfaz web (Streamlit) — Incorporación nueva

Implementa la interfaz gráfica web de la aplicación. A diferencia del modo CLI, no guarda archivos en disco sino que ofrece botones de descarga directa. El script se re-ejecuta por completo ante cada interacción del usuario; el caché de `@st.cache_data` evita recalculación el clustering en cada cambio de parámetro.

Controles del sidebar:

- Cargador de imagen (JPG / PNG).
- Slider de cantidad de colores: 1 a 8.
- Slider de brillo: 0.1 a 3.0.
- Slider de saturación: 0.0 a 3.0.
- Slider de escala de visualización: 10% a 200%.

Funciones principales:

Método y Descripción

`normalizar_texto(texto)` → Elimina tildes y caracteres no ASCII de nombres de archivo

`pil_a_bytes(imagen)` → Convierte imagen PIL a bytes para los botones de descarga

`analizar(imagen, n_colores)` → Extrae colores dominantes; resultado almacenado en caché

`procesar_imagen(img, pal_orig, pal_nueva, ...)` → Aplica paleta + brillo + saturación a la imagen

`crear_comparativa(original, modificada)` → Genera visualización antes/después para la interfaz

Elementos visuales mostrados:

- Imagen original subida por el usuario.
- Paleta dominante extraída (swatches con HEX, temperatura y brillo).
- Las tres paletas alternativas generadas.
- Imagen resultado con la paleta seleccionada aplicada.
- Comparativa antes/después.
- Botones de descarga: imagen procesada, comparativa y paleta visual.

main.py

Punto de entrada mínimo del modo CLI. Instancia PaletaApp y llama a ejecutar().

4.3 Dependencias

Biblioteca Versión Uso

Biblioteca	Versión	Función en el proyecto
Pillow	12.2.0	Carga, edición y guardado de imágenes.
NumPy	2.4.6	Manipulación de matrices y procesamiento de píxeles.
Scikit-learn	1.9.0	Implementación del algoritmo K-Means para la extracción de colores predominantes.
SciPy	1.17.1	Biblioteca de cálculo científico utilizada como dependencia de Scikit-learn.
Joblib	1.5.3	Paralelización de tareas para optimizar el rendimiento de Scikit-learn.
Streamlit	latest	Desarrollo de la interfaz web interactiva de la aplicación.
Narwhals	2.22.0	Dependencia de Streamlit para el procesamiento e interoperabilidad de datos.
ThreadPoolctl	3.6.0	Administración de hilos de ejecución utilizados por Scikit-learn.

5. Resultados

La aplicación fue probada con dos imágenes de referencia:

1. **ladrillitos.jpg**: fotografía de textura con tonos rojizos y terrosos.
2. **psy.jpg**: imagen con colores más variados y saturados.

5.1 Salidas del modo CLI

Para cada ejecución por consola se producen los siguientes archivos:

Carpeta PaletaOriginal/

– PaletaOriginal_Ncolores_nombreimagen.jpg: imagen visual de la paleta extraída con

rectángulos de color, HEX, temperatura y brillo.

– PaletaOriginalFoto_Ncolores_nombreimagen.jpg: imagen original con recuadros

semitransparentes superpuestos.

Carpeta PaletasGuardadas/

– PaletaGuardada_Complementaria_Ncolores_nombreimagen.jpg

– PaletaGuardada_Analogica_Ncolores_nombreimagen.jpg

– PaletaGuardada_Triadica_Ncolores_nombreimagen.jpg

Carpeta ImagenResultado/

– NombrePaleta_Ncolores_nombreimagen.jpg: imagen con la paleta aplicada.

– COMPARATIVA_NombrePaleta_Ncolores_nombreimagen.jpg: imagen lado a lado (original vs. editada) con información de la paleta aplicada.

5.2 Salidas del modo web (Streamlit)

La interfaz web no genera archivos en disco. En su lugar, todos los resultados se visualizan directamente en el navegador y se ofrecen como descargas opcionales mediante botones de descarga directa:

1. Imagen procesada con la paleta aplicada.
2. Imagen comparativa antes/después.
3. Imagen visual de la paleta dominante.

5.3 Ejemplos de resultados

Los archivos disponibles en el repositorio evidencian resultados con 3, 4 y 5 colores sobre ambas imágenes de prueba, con paletas Análoga y Complementaria. El efecto de recolorización es notable especialmente cuando los colores de la paleta difieren significativamente en tono de los colores originales. La interfaz web permite ajustar brillo y saturación en tiempo real, lo que amplía visiblemente el rango de variaciones obtenibles.

6. Pruebas

6.1 Validación de entrada

La aplicación valida la entrada del usuario en los siguientes puntos:

- **Modo CLI — Ruta de imagen:** se verifica con `os.path.isfile()` antes de continuar. Si el archivo no existe, se solicita nuevamente.
- **Modo CLI — Cantidad de colores:** se valida que sea un entero entre 1 y 8. Valores fuera de rango o no numéricos muestran mensajes descriptivos y repiten la solicitud.
- **Modo web — Carga de imagen:** Streamlit restringe los formatos a JPG y PNG en el componente `file_uploader`. Mientras no se suba imagen, la interfaz no muestra resultados.
- **Modo web — Parámetros:** todos los controles son sliders con límites fijos; no es posible ingresar valores fuera de rango.

6.2 Casos de prueba funcionales

Caso Resultado esperado Estado

Caso de prueba	Resultado esperado	Estado
Imagen válida, 3 colores, paleta análoga (CLI).	Se generan correctamente los archivos en las tres carpetas de salida.	Verificado
Imagen válida, 4 colores, paleta análoga (CLI).	Se generan correctamente los archivos en las tres carpetas de salida.	Verificado
Imagen válida, 5 colores, paleta complementaria (CLI).	Se generan correctamente los archivos en las tres carpetas de salida.	Verificado
Carga de una imagen JPG y ajuste de parámetros mediante los controles deslizantes (Web).	Los resultados se actualizan en tiempo real al modificar los parámetros.	Verificado
Descarga de la imagen procesada (Web).	La imagen procesada se descarga correctamente en el navegador.	Verificado
Descarga de la imagen comparativa (Web).	La imagen comparativa (antes/después) se descarga correctamente.	Verificado
La fuente arial.ttf no está disponible.	La aplicación utiliza automáticamente la fuente predeterminada de Pillow.	Manejado
Colores con alto grado de solapamiento luego del clustering.	La fusión automática reduce la cantidad de colores similares en la paleta.	Verificado
Imagen PNG con canal alfa (transparencia).	La imagen se convierte correctamente al formato RGB eliminando el canal alfa.	Verificado

6.3 Casos de borde

- **Imágenes pequeñas:** el método `obtener_paleta_original()` recalcula proporcionalmente el tamaño de los recuadros para que sean visibles sin exceder la imagen.
- **Paletas con muchos colores similares:** `fusionar_colores_similares()` reduce la paleta efectiva, lo que puede resultar en menos colores de los solicitados.
- **Nombres de archivos con caracteres especiales o acentos:** `normalizar_texto()` elimina acentos y caracteres no ASCII mediante `unicodedata.normalize`.
- **Re-ejecución del script en modo web:** gracias a `@st.cache_data`, el clustering no se recalcula si el usuario solo modifica brillo o saturación, garantizando respuesta inmediata.

7. Posibles Mejoras

7.1 Mejora del algoritmo de recolorización

El método actual de recolorización (aplicar_paleta) reemplaza píxeles por proximidad absoluta con un umbral fijo de 47 unidades. Esto puede generar bordes abruptos o colores no reemplazados en regiones con gradientes suaves. Alternativas a explorar:

- **Segmentación semántica** para reemplazar regiones completas en lugar de píxeles individuales.
- **Transferencia de estilo de color** usando el espacio Lab* para una sustitución más natural.
- **Umbral adaptativo** calculado en función de la dispersión de cada clúster.

7.2 Más tipos de armonías

Se podría ampliar GeneradorPaletas con: Tetrádicas (cuatro colores separados 90°);

Split-complementarias (un color y los dos adyacentes a su complementario); Monocromáticas (variaciones

de brillo y saturación del mismo tono).

7.3 Exportación a formatos de diseño

Añadir exportación de las paletas en formatos utilizados por herramientas profesionales: ASE (Adobe Swatch Exchange) para Photoshop/Illustrator; JSON para integración con aplicaciones web; CSS con

variables de color para desarrollo frontend.

7.4 Soporte para múltiples imágenes

Procesar un lote de imágenes en una sola ejecución, generando un informe consolidado de paletas para análisis comparativo. En la interfaz web, esto podría implementarse con un cargador múltiple de archivos.

7.5 Optimización del clustering

Incorporar métodos como el **Método del Codo** (Elbow Method) o el **índice de silueta** para sugerir automáticamente el número óptimo de colores sin que el usuario deba especificarlo.

7.6 Deploy de la interfaz web

La aplicación Streamlit puede publicarse en **Streamlit Community Cloud** conectando el repositorio de GitHub, sin costo y sin configuración de servidor. Esto haría accesible la herramienta a cualquier usuario sin necesidad de instalar Python ni dependencias.

Apéndice: Instrucciones de Uso

Instalación

```
pip install -r requirements.txt
```

Ejecución — Modo consola (CLI)

```
python main.py
```

La aplicación solicitará interactivamente:

1. La ruta de la imagen (ej: ladrillitos.jpg).
2. La cantidad de colores a extraer (entre 1 y 8).
3. La paleta a aplicar (1: Complementaria, 2: Análoga, 3: Triádica).

Los resultados se guardan automáticamente en PaletaOriginal/, PaletasGuardadas/ e ImagenResultado/.

Ejecución — Interfaz web (Streamlit)

```
streamlit run app_streamlit.py
```

El navegador se abre automáticamente en <http://localhost:8501>. El flujo de uso es:

1. Subir una imagen JPG o PNG desde el panel lateral.
2. Ajustar la cantidad de colores, brillo, saturación y escala con los sliders.
3. Visualizar la paleta dominante y las tres paletas alternativas.
4. Descargar la imagen procesada, la comparativa o la paleta visual.